

Exercise: Log Analysis Pipeline

Objective: Build a modular log analysis pipeline that parses, filters, aggregates, and reports on server log data.

Step 1: Project Setup

Objective: Create a new Rust project and organize the module structure.

1. Create a new Rust project:

```
cargo new log_analysis_pipeline  
cd log_analysis_pipeline
```

2. Edit Cargo.toml to include dependencies:

```
[package]  
name = "log_analysis_pipeline"  
version = "0.1.0"  
edition = "2021"  
  
[dependencies]  
serde = { version = "1.0", features = ["derive"] }  
serde_json = "1.0"  
chrono = "0.4"
```

3. Organize the project structure:

1. Create the following files:

- src/log.rs
- src/parsing.rs
- src/filtering.rs
- src/aggregation.rs
- src/threat_detection.rs
- src/output.rs
- src/macros.rs
- src/main.rs

Step 2: Define Data Structures

Objective: Define the core data structures for logs and aggregated data.

1. Edit src/log.rs:

```
use chrono::NaiveDateTime;
use serde::Serialize;
use std::cell::RefCell;
use std::rc::Rc;

#[derive(Debug, Serialize, Clone)]
pub struct LogEntry {
    pub timestamp: NaiveDateTime,
    pub ip_address: String,
    pub endpoint: String,
    pub response_status: u16,
    pub response_time_ms: u32,
}

#[derive(Debug, Default, Serialize)]
pub struct AggregatedData {
    pub requests_per_ip: std::collections::HashMap<String, usize>,
    pub requests_per_endpoint: std::collections::HashMap<String, usize>,
    pub failed_logins: std::collections::HashMap<String, usize>,
}
```

2. Explanation:

- **LogEntry:** Represents a single log entry with timestamp, IP address, endpoint, response status, and response time.
- **AggregatedData:** Stores aggregated statistics (requests per IP, requests per endpoint, failed logins).

Step 3: Implement the Parsing Module

Objective: Parse log entries from a CSV file.

1. Edit src/parsing.rs:

```
use crate::log::LogEntry;
use chrono::NaiveDateTime;
use std::fs::File;
use std::io::{BufRead, BufReader};
use std::path::Path;

pub fn parse_logs(file_path: &str) -> Result<Vec<LogEntry>, String> {
    let path = Path::new(file_path);
    let file = File::open(path).map_err(|e| e.to_string());
    let reader = BufReader::new(file);

    let mut logs = Vec::new();
    for line in reader.lines() {
        let line = line.map_err(|e| e.to_string());
        let fields: Vec<&str> = line.split(',').collect();
        if fields.len() != 5 {
            continue; // Skip malformed lines
        }
        let timestamp = NaiveDateTime::parse_from_str(fields[0], "%Y-%m-%dT%H:%M:%S")
            .map_err(|e| e.to_string());
        let log = LogEntry {
            timestamp,
            ip_address: fields[1].to_string(),
            endpoint: fields[2].to_string(),
            response_status: fields[3].parse().map_err(|e| e.to_string()),
            response_time_ms: fields[4].parse().map_err(|e| e.to_string()),
        };
        logs.push(log);
    }
}
```

2. Explanation:

- The parse_logs function reads a CSV file and converts each line into a LogEntry.
- Uses chrono::NaiveDateTime to parse timestamps.

Step 4: Implement the Filtering Module

Objective: Filter out invalid log entries.

1. Edit src/filtering.rs:

```
use crate::log::LogEntry;

pub fn filter_valid_logs(logs: Vec<LogEntry>) -> Vec<LogEntry> {
    logs.into_iter()
        .filter(|log| log.response_status < 600) // Filter out invalid status codes
        .collect()
}
```

2. **Explanation:**

- The filter_valid_logs function removes entries with invalid HTTP status codes (≥ 600).

Step 5: Implement the Aggregation Module

Objective: Aggregate log data into statistics.

1. Edit src/aggregation.rs:

```
use crate::log::{AggregatedData, LogEntry};

pub fn aggregate_data(logs: &[LogEntry]) -> AggregatedData {
    let mut data = AggregatedData::default();
    for log in logs {
        *data.requests_per_ip.entry(log.ip_address.clone()).or_insert(0) += 1;
        *data.requests_per_endpoint.entry(log.endpoint.clone()).or_insert(0) += 1;
        if log.response_status == 401 {
            *data.failed_logins.entry(log.ip_address.clone()).or_insert(0) += 1;
        }
    }
    data
}
```

2. **Explanation:**

- The aggregate_data function populates AggregatedData with counts of requests per IP, requests per endpoint, and failed logins.

Step 6: Implement the Threat Detection Module

Objective: Detect potential security threats based on failed login attempts.

1. Edit src/threat_detection.rs:

```
use crate::log::AggregatedData;

pub fn detect_threats(data: &AggregatedData, threshold: usize) -> Vec<String> {
    data.failed_logins
        .iter()
        .filter(|(&_, &count)| count >= threshold)
        .map(|(ip, _)| ip.clone())
        .collect()
}
```

2. Explanation:

- The detect_threats function identifies IPs with failed login attempts exceeding a specified threshold.

Step 7: Implement the Output Module

Objective: Generate a JSON report of the aggregated data.

1. Edit src/output.rs:

```
use crate::log::AggregatedData;
use serde_json;
use std::fs::File;
use std::io::Write;

pub fn write_report(data: &AggregatedData, file_path: &str) -> Result<(), String> {
    let report = serde_json::to_string_pretty(data).map_err(|e| e.to_string())?;
    let mut file = File::create(file_path).map_err(|e| e.to_string())?;
    file.write_all(report.as_bytes()).map_err(|e| e.to_string())?;
    Ok(())
}
```

2. Explanation:

- The write_report function serializes AggregatedData to JSON and writes it to a file.

Step 8: Implement Macros for Logging

Objective: Create macros for logging errors and debug information.

1. Edit src/macros.rs:

```
#[macro_export]
macro_rules! log_error {
    ($($arg:tt)*) => {
        eprintln!("[ERROR] {}", format_args!($($arg)*));
    };
}

#[macro_export]
macro_rules! log_debug {
    ($($arg:tt)*) => {
        println!("[DEBUG] {}", format_args!($($arg)*));
    };
}
```

2. Explanation:

- **log_error!** and **log_debug!** macros simplify logging errors and debug messages.

Step 9: Implement the Main Pipeline

Objective: Integrate all modules into a cohesive pipeline.

1. Edit src/main.rs:

```
mod log;
mod parsing;
mod filtering;
mod aggregation;
mod threat_detection;
mod output;
mod macros;

use log::AggregatedData;
use parsing::parse_logs;
use filtering::filter_valid_logs;
use aggregation::aggregate_data;
use threat_detection::detect_threats;
use output::write_report;

fn main() {
    let logs = parse_logs("server_logs.csv").unwrap_or_else(|e| {
        log_error!("Failed to parse logs: {}", e);
        Vec::new()
    });
    let valid_logs = filter_valid_logs(logs);
    let data = aggregate_data(&valid_logs);
    let threats = detect_threats(&data, 3); // Threshold: 3 failed logins
    log_debug!("Potential threats: {:?}", threats);
    write_report(&data, "report.json").unwrap_or_else(|e| {
        log_error!("Failed to write report: {}", e);
    });
}
```

2. Explanation:

- The main function orchestrates the pipeline: parsing, filtering, aggregating, detecting threats, and writing the report.

Step 10: Create a Sample Log File

Objective: Test the pipeline with sample data.

1. Create server_logs.csv in the project root:

```
2026-01-14T12:34:56,192.168.1.1,/api/login,401,120
2026-01-14T12:35:01,192.168.1.2,/api/data,200,85
2026-01-14T12:35:02,192.168.1.1,/api/login,401,95
2026-01-14T12:35:03,192.168.1.3,/api/data,200,110
```

Step 11: Run and Test the Pipeline

Objective: Execute the pipeline and verify the output.

1. Run the project:

```
cargo run
```

Expected Output:

2. A report.json file with aggregated statistics:

```
{
  "requests_per_ip": {
    "192.168.1.1": 2,
    "192.168.1.2": 1,
    "192.168.1.3": 1
  },
  "requests_per_endpoint": {
    "/api/login": 2,
    "/api/data": 2
  },
  "failed_logins": {
    "192.168.1.1": 2
  }
}
```


Step 12: Extensions (Optional)

Objective: Enhance the pipeline with additional features.

1. **Spatial Tree for IP Geolocation:** (joke)
 - Use a spatial tree to visualize IP geolocation data.
2. **Multithreading:**
 - Use Arc<T> and threads to parallelize log processing.
3. **Custom Derive Macro:**
 - Implement a custom derive macro for log entry validation.